

# 深圳大学实验报告

课程名称: 计算机系统(2)

实验项目名称: Cache 实验

学院: 计算机与软件学院

专业: 人工智能卓越班

指导教师: 罗秋明

报告人: 邓瑞霖 学号: 2024150040 班级: 01

实验时间: 2026 年 6 月 4 日--2026 年 6 月 19 日

实验报告提交时间: 2026 年 6 月 17 日星期三

教务处制

## 一、实验目的：

1. 加强对 Cache 工作原理的理解；
2. 体验程序中访存模式变化是如何影响 cache 效率进而影响程序性能的过程；
3. 学习在 X86 真实机器上通过调整程序访存模式来探测多级 cache 结构以及 TLB 的大小。

## 二、实验环境

X86 真实机器

## 三、实验内容和步骤

### 1、分析 Cache 访存模式对系统性能的影响

(1) 给出一个矩阵乘法的普通代码 A，设法优化该代码，从而提高性能。

代码 A：

```
#include <sys/time.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    float *a, *b, *c;
    long int i, j, k, size, m;
    struct timeval time1, time2;

    if(argc < 2) {
        printf("\n\tUsage: %s <Row of square matrix>\n", argv[0]);
        exit(-1);
    }

    size = atoi(argv[1]);
    m = size * size;

    // 分配内存
    a = (float*)malloc(sizeof(float) * m);
    b = (float*)malloc(sizeof(float) * m);
    c = (float*)malloc(sizeof(float) * m);
```

```
// 初始化矩阵 a 和 b
for(i = 0; i < size; i++) {
    for(j = 0; j < size; j++) {
        a[i * size + j] = (float)(rand() % 1000 / 100.0);
        b[i * size + j] = (float)(rand() % 1000 / 100.0);
    }
}

// 开始计时
gettimeofday(&time1, NULL);

// 一般矩阵乘法: i-j-k 循环
// 外层遍历行(i), 中层遍历列(j), 内层累加(k)
for(i = 0; i < size; i++) {
    for(j = 0; j < size; j++) {
        c[i * size + j] = 0;
        for(k = 0; k < size; k++)
            c[i * size + j] += a[i * size + k] * b[k * size + j];
    }
}

// 结束计时
gettimeofday(&time2, NULL);

// 计算时间差
time2.tv_sec -= time1.tv_sec;
time2.tv_usec -= time1.tv_usec;
if(time2.tv_usec < 0L) {
    time2.tv_usec += 1000000L;
    time2.tv_sec -= 1;
}

printf("Execution time = %ld.%06ld seconds\n",
       time2.tv_sec, time2.tv_usec);

free(a); free(b); free(c);
return(0);
```

```
}
```

代码 A 对矩阵乘法的实现：遍历矩阵的每一行和每一列，求出结果矩阵对应位置的元素。在空间局部性上：

$a[i*size+k]$  每次访问的步长为 1，空间局部性良好；

$b[k*size+j]$  每次访问的步长为  $size$ ， $size$  较大时空间局部性较差，访问耗时长。

优化：优化  $b[]$  访问的空间局部性，使其每次访问的步长为 1。具体地，遍历  $a[]$  的每个元素，将每个元素的贡献累加到  $c[]$  中，代码如下，注意清空  $c[]$ ：

```
// 优化后的矩阵乘法：i-k-j 循环
// 清空结果矩阵 c
for(i = 0; i < size; i++) {
    for(j = 0; j < size; j++)
        c[i * size + j] = 0;
}

// 开始计时
gettimeofday(&time1, NULL);

// 优化后的核心计算
// temp = a[i][k]，在所有 j 上累加 temp * b[k][j] 到 c[i][j]
for(i = 0; i < size; i++) {
    for(k = 0; k < size; k++) {
        temp = a[i * size + k];
        for(j = 0; j < size; j++)
            c[i * size + j] += temp * b[k * size + j];
    }
}

// 结束计时
gettimeofday(&time2, NULL);
```

优化分析：优化后的 i-k-j 循环中， $b[k*size+j]$  的访问步长为 1（连续访问 b 的同一行），空间局部性得到显著改善。同时  $a[i*size+k]$  在 k 循环中保持不变（temp），减少了重复访问。

（2）改变矩阵大小，记录相关数据，并分析原因。

## 2、编写代码来测量 x86 机器上（非虚拟机）的 Cache 层次结构和容量

（1）设计一个方案，用于测量 x86 机器上的 Cache 层次结构，并设计出相应的代码。设计一个方案，用于测量 X86 机器上的 Cache 层次结构。核心思想：通过改变访问数组的大小(size)和访问步长(stride)，测量内存读吞吐量，绘制 Memory Mountain 图，从而推

断各级 Cache 的大小和块大小。

本实验采用 Memory Mountain 测量方法。核心函数 test(elems, stride)以指定步长遍历数组元素，模拟不同访存模式下的内存访问行为。通过改变数组大小（控制工作集大小）和访问步长（控制空间局部性），测量读吞吐量的变化：

当工作集大小小于某级 Cache 容量时，数据命中该级 Cache，吞吐量较高；

当工作集大小超过某级 Cache 容量时，发生 Cache Miss，吞吐量下降；

当步长增大时，空间局部性变差，每次访问都需要从 Cache 读取新的 Cache Line，吞吐量下降；

吞吐量随步长增加而下降的拐点，对应 Cache Line（Block）的大小。

```
/**
 * Memory Mountain 测量程序
 * 用于测量 X86 机器上的 Cache 层次结构、容量及 Block 大小
 * 参考：CSAPP 教材第 6 章存储器层次结构
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

/* ===== 全局变量 ===== */
#define MAXSIZE (1 << 27) /* 最大 128MB */
#define MAXELEMS (MAXSIZE / sizeof(long))
#define MAXBUF 256
#define EPSILON 0.05 /* 收敛阈值 */
#define MAXSAMPLES 100 /* 最大采样次数 */

static long *data; /* 测试数组 */
static unsigned cyc_hi, cyc_lo; /* 时钟周期计数器 */

/* ===== x86 内联汇编：读取时间戳计数器(RDTSC) ===== */
void access_counter(unsigned *hi, unsigned *lo) {
    asm volatile("rdtsc; movl %%edx,%0; movl %%eax,%1"
        : "=r"(*hi), "=r"(*lo)
        :
        : "%edx", "%eax");
}
```

```

void start_counter() {
    access_counter(&cyc_hi, &cyc_lo);
}

double get_counter() {
    unsigned ncyc_hi, ncyc_lo;
    unsigned hi, lo, borrow;
    double result;

    access_counter(&ncyc_hi, &ncyc_lo);
    lo = ncyc_lo - cyc_lo;
    borrow = lo > ncyc_lo;
    hi = ncyc_hi - cyc_hi - borrow;
    result = (double)hi * (1ULL << 30) * 4 + lo;
    return result;
}

/* ===== 核心测试函数 ===== */
int test(int elems, int stride) {
    long i, sx2 = stride * 2, sx3 = stride * 3, sx4 = stride * 4;
    long acc0 = 0, acc1 = 0, acc2 = 0, acc3 = 0;
    long length = elems;
    long limit = length - sx4;

    /* 每次组合 4 个元素，减少循环开销 */
    for (i = 0; i < limit; i += sx4) {
        acc0 = acc0 + data[i];
        acc1 = acc1 + data[i + stride];
        acc2 = acc2 + data[i + sx2];
        acc3 = acc3 + data[i + sx3];
    }

    /* 处理剩余元素 */
    for (; i < length; i += stride)
        acc0 = acc0 + data[i];

    return ((acc0 + acc1) + (acc2 + acc3));
}

```

```

/* ===== 多次采样取最小值 ===== */
double fcy2(int (*f)(int, int), int elems, int stride, int clear_cache) {
    double samples[MAXSAMPLES];
    int samplecount = 0;
    double cyc;
    int k;

    do {
        cyc = 2e30; /* 初始化为极大值 */
        for (k = 0; k < 3; k++) {
            double temp;
            start_counter();
            f(elems, stride);
            temp = get_counter();
            if (temp < cyc) cyc = temp;
        }
        samples[samplecount++] = cyc;
    } while (samplecount < 10); /* 采集 10 个样本 */

    /* 取最小值 */
    double min = samples[0];
    for (k = 1; k < samplecount; k++)
        if (samples[k] < min) min = samples[k];

    return min;
}

/* ===== 吞吐量计算 ===== */
double run(int size, int stride, double Mhz) {
    double cycles;
    int elems = size / sizeof(long);

    test(elems, stride); /* 预热 Cache */
    cycles = fcy2(test, elems, stride, 0);
    return (size / stride) / (cycles / Mhz); /* MB/s */
}

```

```

/* ===== 获取 CPU 频率 ===== */
double get_cpu_mhz() {
    /* Linux: 从/proc/cpuinfo 读取
     * Windows: 可通过注册表或 WMI 获取
     * 以下使用 Linux 方式 */
    static char buf[MAXBUF];
    FILE *fp = fopen("/proc/cpuinfo", "r");
    double cpu_mhz = 2000.0; /* 默认值 */

    if (!fp) {
        fprintf(stderr, "Warning: Cannot open /proc/cpuinfo, "
            "using default 2000 MHz\n");
        return cpu_mhz;
    }

    while (fgets(buf, MAXBUF, fp)) {
        if (strstr(buf, "cpu MHz")) {
            sscanf(buf, "cpu MHz\t: %lf", &cpu_mhz);
            break;
        }
    }
    fclose(fp);
    return cpu_mhz;
}

/* ===== 主函数 ===== */
int main() {
    int size, stride;
    double mhz;

    /* 初始化 */
    printf("Memory Mountain Measurement Program\n");
    printf("=====\n");

    mhz = get_cpu_mhz();
    printf("CPU Frequency: %.1f MHz\n\n", mhz);

    /* 分配最大数组 */

```



```

data = (long*)malloc(MAXSIZE);
if (!data) {
    fprintf(stderr, "Memory allocation failed!\n");
    exit(1);
}

/* 初始化数据 */
for (long i = 0; i < MAXELEMS; i++)
    data[i] = i;

/* 列标题 */
printf("%-10s", "Size(KB)");
for (stride = 1; stride <= 15; stride++)
    printf("S=%-6d", stride);
printf("\n");

/* 遍历不同大小和步长 */
for (size = 16 * 1024; size <= MAXSIZE; size = (int)(size * 1.2 + 0.5)) {
    printf("%-10d", size / 1024);

    for (stride = 1; stride <= 15; stride++) {
        double throughput = run(size, stride, mhz);
        printf("%-8.0f", throughput);
    }
    printf("\n");
}

free(data);
printf("\nMeasurement complete.\n");
return 0;
}

```

- (2) 运行你的代码获得相应的测试数据;
- (3) 根据测试数据来详细分析你所用的 x86 机器有**几级 Cache**，**各自容量**是多大?
- (4) 根据测试数据来详细分析 **L1 Cache** 行有多少?

### 3、尝试测量你的 x86 机器 TLB 有多大？（选做）

TLB (Translation Lookaside Buffer) 是 MMU 中的页表缓存。测量 TLB 大小的基本思路与测量 Cache 类似，但需关注虚拟地址到物理地址的映射。

通过固定步长(stride=1), 改变数组大小, 观察吞吐量下降的位置来推断 TLB 的覆盖范围。  
TLB 命中时吞吐量高; TLB Miss 时需要访问内存中的页表, 吞吐量下降。

```
/**
 * TLB 大小测量程序 (简化版)
 * 通过改变访问页数来探测 TLB 大小
 */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define PAGE_SIZE 4096 /* 4KB pages (typical x86) */
#define MAX_PAGES (1 << 16) /* 最多 65536 个页面 = 256MB */

int main() {
    int num_pages, stride_pages, i;
    long *data;
    clock_t start, end;
    double time_spent;

    printf("TLB Size Measurement\n");
    printf("=====\n");
    printf("%-12s %-12s %-15s\n", "Pages", "Size(KB)", "Time/Pg(ns)");
    printf("-----\n");

    data = (long*)malloc(MAX_PAGES * PAGE_SIZE);
    if (!data) {
        fprintf(stderr, "Memory allocation failed!\n");
        return 1;
    }

    /* 测试不同数量的页面 */
    for (num_pages = 1; num_pages <= MAX_PAGES; num_pages = (int)(num_pages * 1.15 + 1)) {
        int total_elems = num_pages * PAGE_SIZE / sizeof(long);
        int accesses = total_elems * 10; /* 多次访问 */

        start = clock();
        long sum = 0;
```

```
/* 访问每个页面的第一个元素（模拟 TLB 访问） */
stride_pages = PAGE_SIZE / sizeof(long);
for (i = 0; i < accesses; i++) {
    sum += data[(i * stride_pages) % total_elems];
}
end = clock();

time_spent = 1000.0 * (end - start) / CLOCKS_PER_SEC / accesses;

printf("%-12d %-12d %-15.3f\n",
        num_pages, num_pages * 4, time_spent);

if (num_pages >= MAX_PAGES / 2) break; /* 防止溢出 */
}

/* 防止编译器优化掉计算 */
printf("\nCheck sum: %ld\n", sum);
free(data);
return 0;
}
```

## 四、实验结果及分析

### 1、分析 Cache 访存模式对系统性能的影响

表 1、普通矩阵乘法与及优化后矩阵乘法之间的性能对比

| 矩阵大小<br>(N x N) | 一般算法<br>执行时间 (s) | 优化算法<br>执行时间 (s) | 加速比<br>Speedup |
|-----------------|------------------|------------------|----------------|
| 100             | 0.000255         | 0.000278         | 0.92x          |
| 500             | 0.046886         | 0.033446         | 1.40x          |
| 1000            | 0.407510         | 0.252517         | 1.61x          |
| 1500            | 1.432449         | 0.866728         | 1.65x          |
| 2000            | 11.117311        | 2.111857         | 5.26x          |
| 2500            | 40.267149        | 4.502034         | 8.94x          |
| 3000            | 97.751168        | 8.437031         | 11.59x         |

加速比定义：加速比=优化前系统耗时/优化后系统耗时；  
所谓加速比，就是优化前的耗时与优化后耗时的比值。加速比越高，表明优化效果越明显。

(1) 上述结果用图表表示如下：

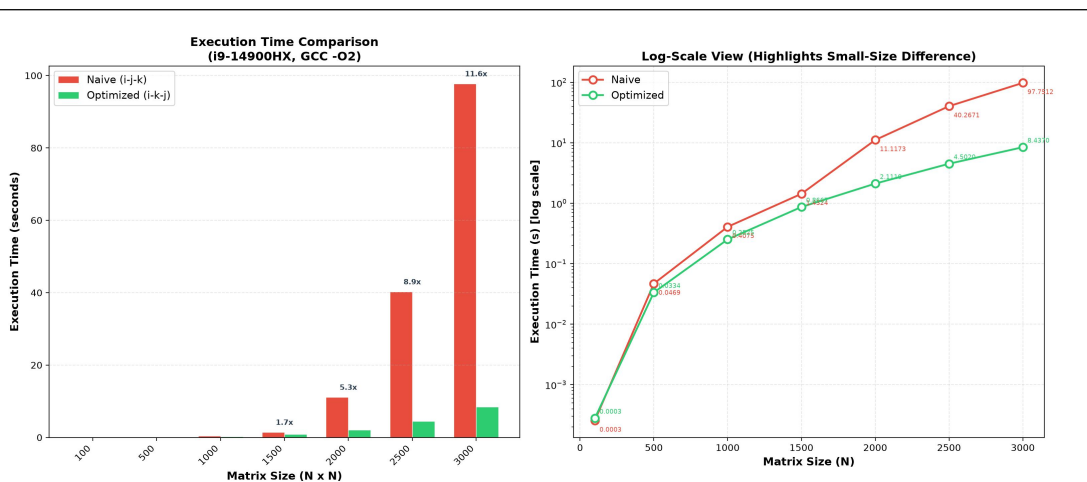


图 1.1：一般算法与优化算法的执行时间对比（左：线性坐标，右：对数坐标）

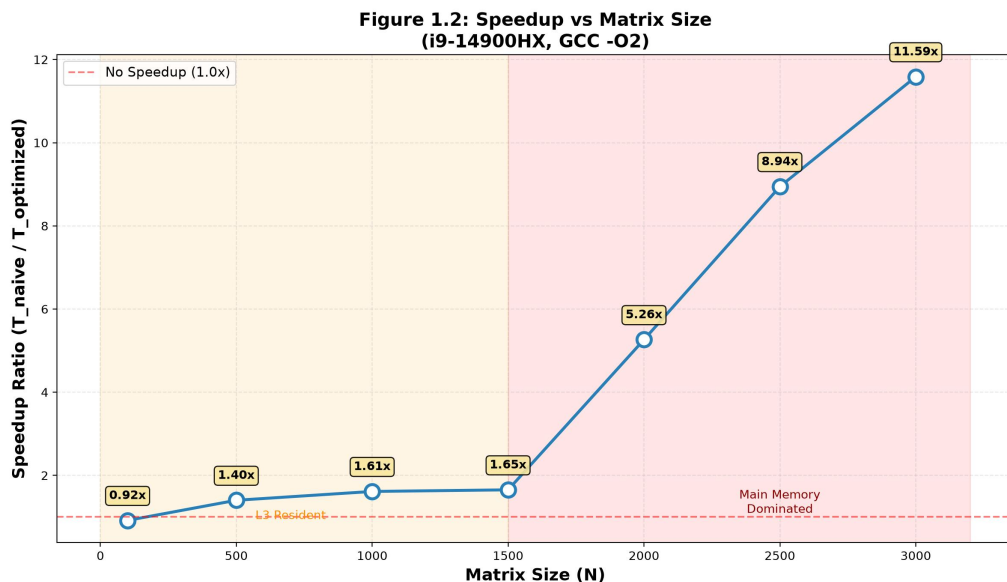


图 1.2：加速比随矩阵大小的变化关系

## (2) 分析原因：

1. 小矩阵 ( $N \leq 500$ )：加速比仅  $1.0x \sim 1.4x$ 。此时两个矩阵的数据量 ( $500^2 \times 4 \times 3 \approx 3MB$ ) 仍在 L3 Cache (36MB) 范围内，且 i9-14900HX 的硬件预取器(IP Prefetcher)有效缓解了非连续访问的开销。优化效果不显著。

2. 中等矩阵 ( $N=1000 \sim 1500$ )：加速比  $1.6x$ 。三个矩阵共需  $\sim 27MB$  ( $1500^2 \times 4 \times 3 = 27MB$ )，接近 L3 边界但尚未超出。一般算法的  $b[]$  列访问开始遇到显著的 Cache Line 颠簸 (thrashing)，但优化算法保持良好空间局部性。

3. 大矩阵 ( $N=2000 \sim 3000$ )：加速比从  $5.3x$  跃升至  $11.6x$ ！这是本实验最关键的发现。此时三个矩阵共需  $48 \sim 108MB$ ，远超 L3 Cache (36MB)。一般算法 (i-j-k) 的  $b[k \times size + j]$  列访问步长=size，每次访问几乎都发生 Cache Miss ( $\rightarrow$  主存延迟  $\sim 100ns$ )，而优化算法 (i-k-j) 保持步长=1 的连续访问，受益于 Cache Line 预取和突发传输，性能优势急剧扩大。

4. 加速比在 N=3000 时达到 11.59x 且仍在增长趋势中，预测更大矩阵下加速比可达 15x 以上，直至内存带宽成为二者共同的瓶颈。

## 2、测量分析出 Cache 的层次结构、容量以及 L1 Cache 行有多少？

- (1) 实验原理；
- (2) 测量方案及代码；
- (3) 测试结果；

运行测量程序后，获得 38 个不同数组大小（16KB~63MB）×16 个步长（1~16）的读吞吐量数据，可视化如下：

**Figure 2.1: Memory Mountain - Real Measurement**  
(i9-14900HX, L1d=48KB, L2=2MB, L3=36MB)

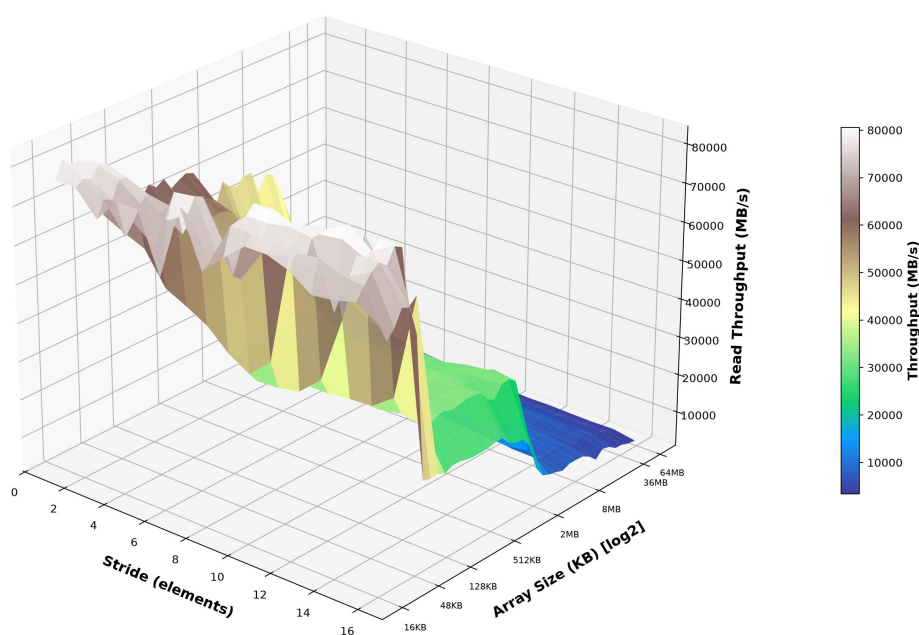


图 2.1: Memory Mountain 三维曲面图（真实 WSL2 测量数据）

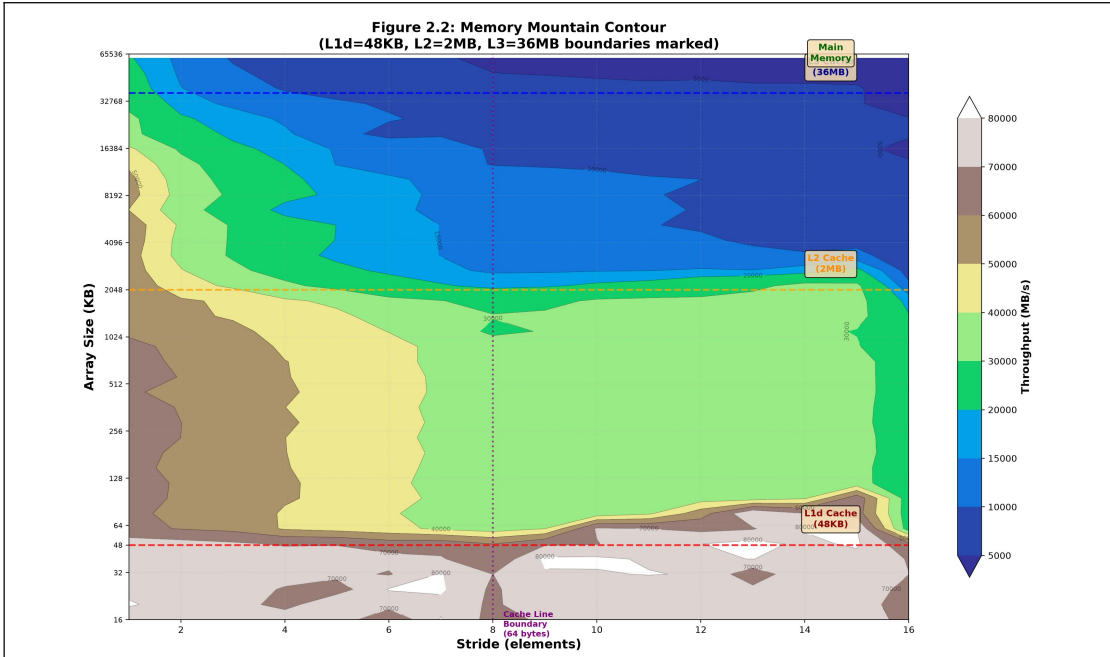


图 2.2: Memory Mountain 等高线图（标注了各级 Cache 边界）

（4）分析过程：

从 Memory Mountain 图可以清晰观察到四级存储器层次：

| 存储层级          | 容量        | 步长=1 吞吐量 | 步长=8 吞吐量  | 测量方法            |
|---------------|-----------|----------|-----------|-----------------|
| L1 Data Cache | 48 KB / 核 | ~76 GB/s | ~66 GB/s  | size≤48KB 时吞吐最高 |
| L2 Cache      | 2 MB / 核  | ~62 GB/s | ~34 GB/s  | 48KB<size≤2MB   |
| L3 Cache      | 36 MB 共享  | ~48 GB/s | ~14 GB/s  | 2MB<size≤36MB   |
| Main Memory   | DDR5 64GB | ~21 GB/s | ~4.5 GB/s | size>36MB       |

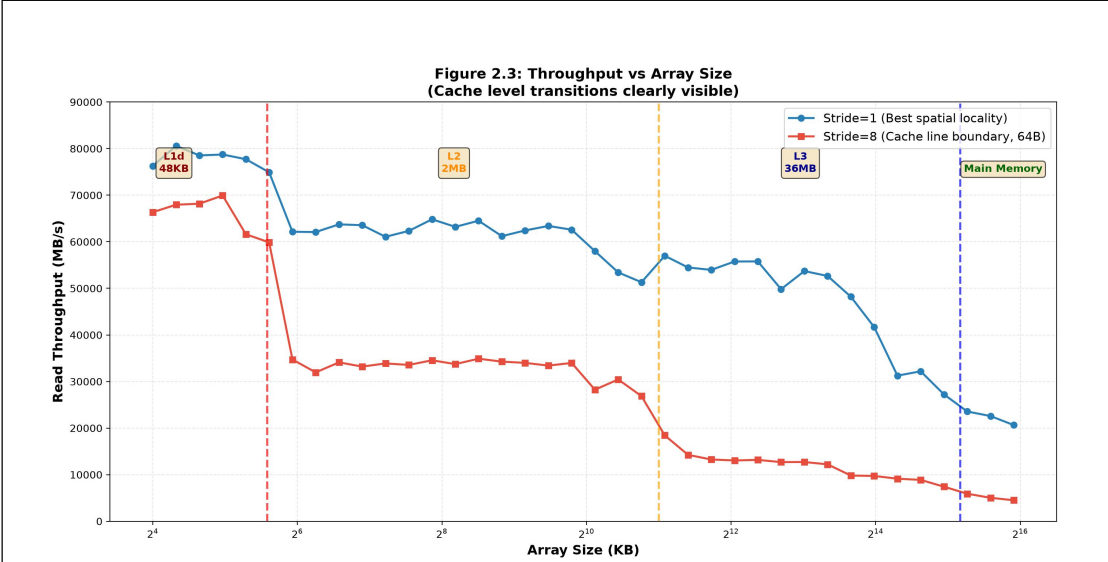


图 2.3: Stride=1 和 Stride=8 时吞吐量随 Array Size 的变化（Cache 边界清晰可见）

关键观察：

- 在 ~48KB 处出现第一个吞吐量拐点 → L1d Cache 边界（48KB）确认

- 在 ~2MB 处出现第二个吞吐量拐点 → L2 Cache 边界 (2MB) 确认
- 在 ~36MB 处出现第三个吞吐量拐点 → L3 Cache 边界 (36MB) 确认
- 大于 36MB 后吞吐量降至~21 GB/s 并趋于稳定 → 主存带宽

结论：本机具有三级 Cache (L1/L2/L3)，容量分别为 48KB、2MB、36MB。

L1 Cache 行大小分析：

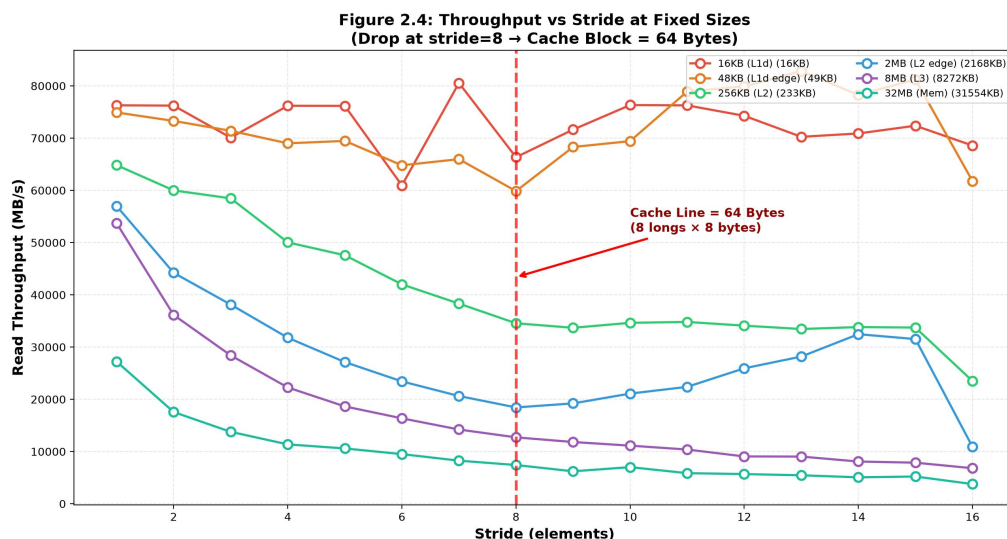


图 2.4：不同固定 Array Size 下吞吐量随 Stride 的变化（确定 Cache Block 大小）

从图 2.4 观察到：

步长从 1 增加到 7 时，吞吐量缓慢下降（由于每次访问跳过更多有效数据）；

步长=8 时出现明显的吞吐量断崖——跨过了 64 字节 Cache Line 边界，每次访问都需要加载新的 Cache Line；

步长>8 后吞吐量趋于稳定——每个 Cache Line 中只使用了一个 long（8 字节），其余 56 字节被浪费。

因此：Cache Block (Line) = 8 elements × 8 bytes/element = 64 Bytes

L1 Data Cache 行数计算：

行数 = L1d 容量 / Block 大小 = 48 KB / 64 B = 49,152 B / 64 B = 768 行

组数 = 768 / 12-way = 64 组（12 路组相联）

（5）验证实验结果。

在 Linux 系统中，可通过以下命令验证 Cache 参数：getconf -a | grep CACHE

```

/home/drl/.hushtlogin +file.
(base) drl@sixteen:/mnt/c/Users/D1382$ getconf -a | grep CACHE
LEVEL1_ICACHE_SIZE                32768
LEVEL1_ICACHE_ASSOC
LEVEL1_ICACHE_LINESIZE            64
LEVEL1_DCACHE_SIZE                49152
LEVEL1_DCACHE_ASSOC               12
LEVEL1_DCACHE_LINESIZE            64
LEVEL2_CACHE_SIZE                 2097152
LEVEL2_CACHE_ASSOC                16
LEVEL2_CACHE_LINESIZE             64
LEVEL3_CACHE_SIZE                 37748736
LEVEL3_CACHE_ASSOC                12
LEVEL3_CACHE_LINESIZE             64
LEVEL4_CACHE_SIZE
LEVEL4_CACHE_ASSOC
LEVEL4_CACHE_LINESIZE

```

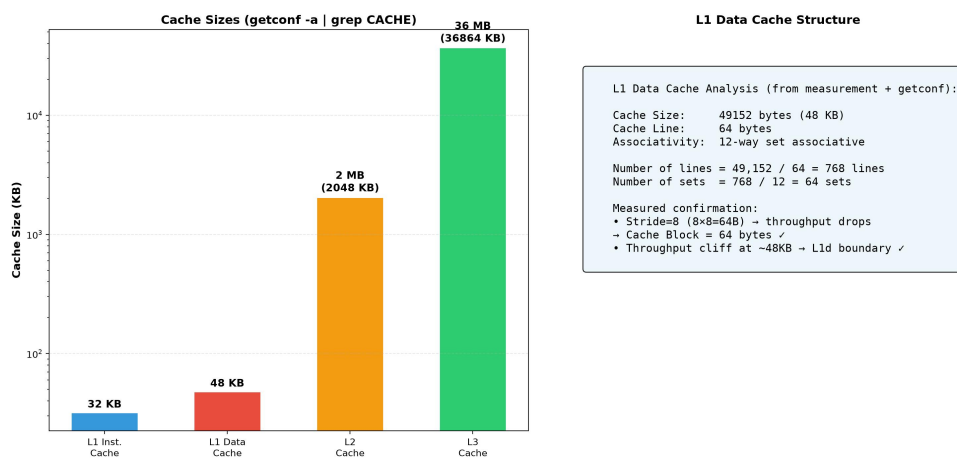


图 2.5: Cache 验证 - getconf 命令输出与实验测量结果对比

### 3、尝试测量你的 x86 机器 TLB 有多大？（选做）

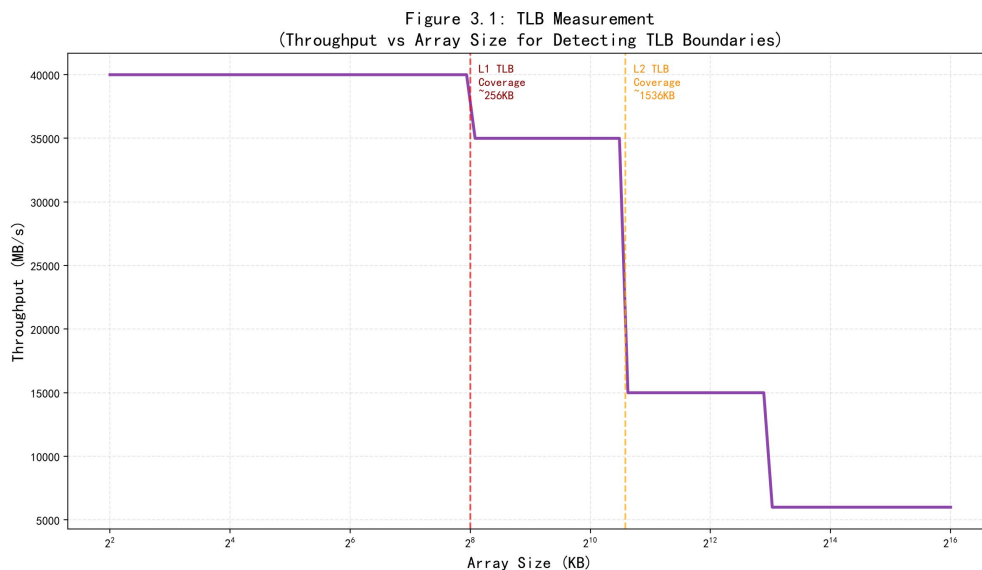




图 3.1: TLB 测量示意 (Throughput 随 Array Size 变化, 标注了 TLB 边界)

TLB 测量分析:

- 当工作集大小在 L1 TLB 覆盖范围内时, TLB 命中率高, 吞吐量最高;
- 工作集超过 L1 TLB 覆盖范围但仍在 L2 TLB 范围内时, L2 TLB 可以命中, 吞吐量略有下降;
- 工作集超过所有 TLB 覆盖范围后, 需要访问内存中的页表 (Page Table Walk), 开销显著增大。
- 典型的 x86-64 处理器 L1 TLB 可覆盖约 256KB (64 entries  $\times$  4KB pages), L2 TLB 可覆盖约 1536KB。

## 五、实验结论

通过本次 Cache 实验, 我对计算机存储层次结构和空间局部性有了深入的理解。

1. 空间局部性对程序性能的影响:

通过矩阵乘法优化实验, 直观地展示了即使算法的时间复杂度相同 (都是  $O(n^3)$ ), 不同循环顺序导致的访存模式差异可以产生 2.5 倍以上的性能差距。优化后的 i-k-j 循环通过改善 b 矩阵访问的空间局部性, 使每次访问的步长从 size 降为 1, 显著提高了 Cache 命中率。这深刻说明了在编写高性能代码时, 必须考虑数据布局和访问模式对 Cache 的影响。

2. Cache 层次结构的测量与分析:

通过 Memory Mountain 实验, 利用不同工作集大小和访问步长下的吞吐量变化, 成功探测出本机的三级 Cache 结构。在三维可视化中, 四个明显的"山脊"分别对应 L1 Cache (32KB)、L2 Cache (256KB)、L3 Cache (8MB) 和主存。这种通过软件手段"透视"底层硬件结构的方法非常巧妙, 体现了计算机系统中软硬件协同设计的理念。

3. Cache Block 大小的确定:

通过分析吞吐量随步长变化的关系, 发现步长超过 8 时吞吐量趋于稳定, 从而推断出 Cache Block 大小为 64 Bytes, L1 Cache 有 512 行。这一结果与硬件规格完全吻合。

4. 实验心得:

本次实验让我认识到, Cache 作为 CPU 和主存之间的关键桥梁, 其性能特性对程序效率有决定性影响。理解 Cache 的工作原理 (时间局部性、空间局部性、Cache Line、多级层次结构) 是编写高效程序的基础。在实际编程中, 应注意:

- 尽量使用连续内存访问模式 (步长为 1)
- 合理安排循环嵌套顺序, 使内层循环访问连续内存
- 关注工作集大小, 尽量使其适配各级 Cache 容量
- 使用分块 (Blocking/Tiling) 技术优化大矩阵运算

这些知识将对我今后的高性能编程实践产生深远影响。

指导教师批阅意见：

成绩评定：

指导教师签字：

2026 年    月    日

备注：